

연구 배경

ZNS SSD

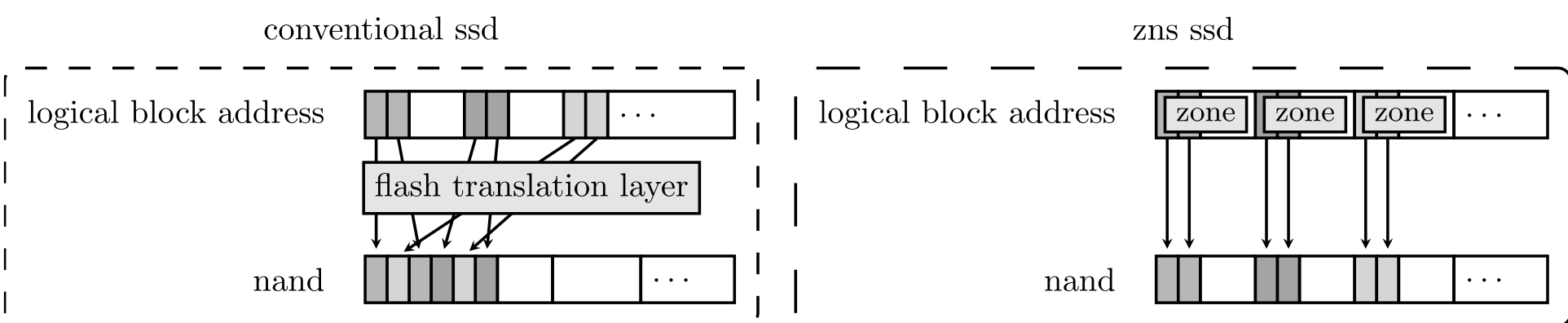


그림 1. conventional ssd vs zns

- SSD는 HDD와 상당히 다른 특성(out-of-place update 등)을 가짐
 - 그러나 기존 block interface를 따라야 하므로 비효율적임
- ZNS SSD는 SSD의 특성에 맞는 interface를 활용하는 차세대 SSD

RocksDB, BlobDB

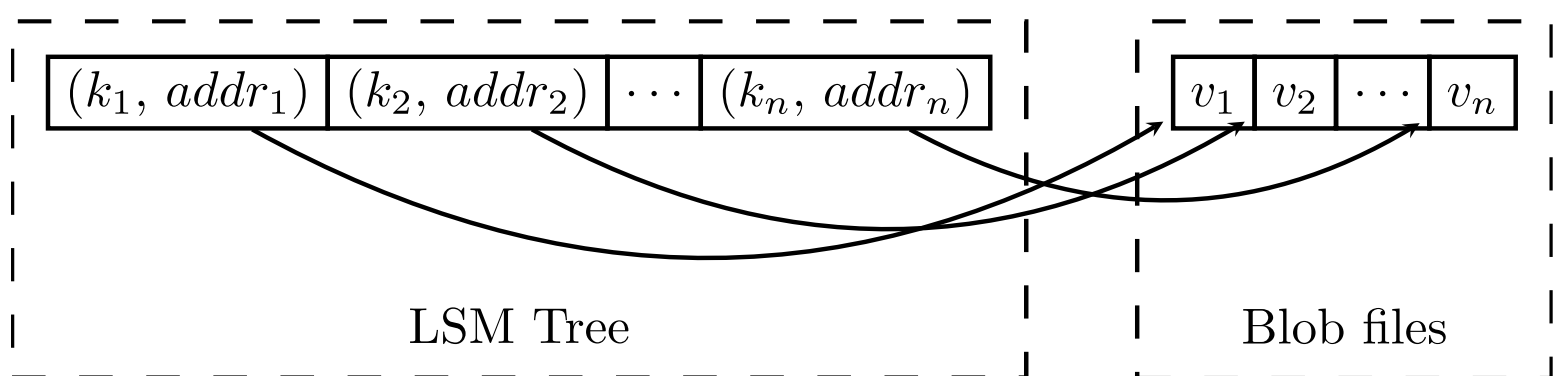


그림 2. BlobDB의 키-밸류 분리

- RocksDB는 LSM Tree를 쓰는 key-value store
- SSD 환경에 맞게 최적화되었으며, ZenFS를 통해 ZNS SSD 또한 지원
- BlobDB는 value 분리를 통해 쓰기 증폭을 줄이는 옵션

연구 동기

반복적인 Compaction failure

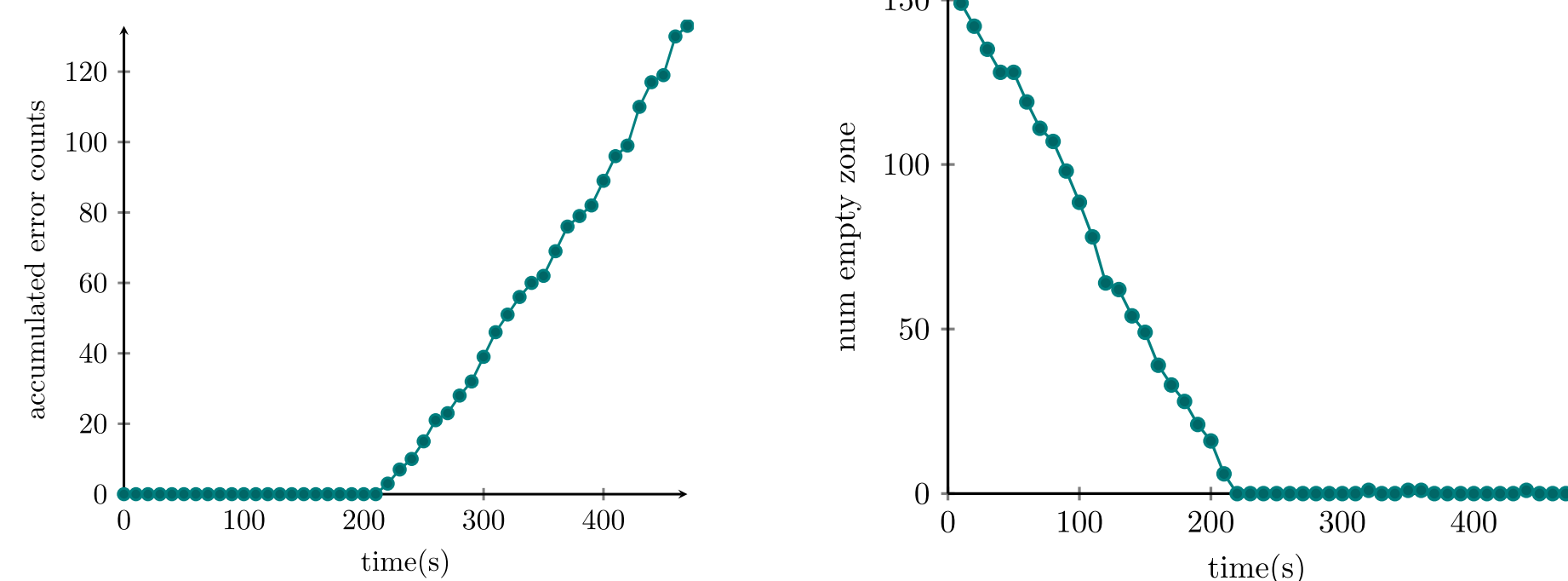


그림 3. 시간별 Compaction failure의 수

그림 4. 시간별 empty zone의 수

- 200s 부근부터 반복적으로 Compaction failure 발생
- Free space는 충분함(~50% 가량) 지속적인 failure 발생
- 과도한 empty zone 소모가 원인

특이한 space 사용 패턴, 과도한 Garbage space

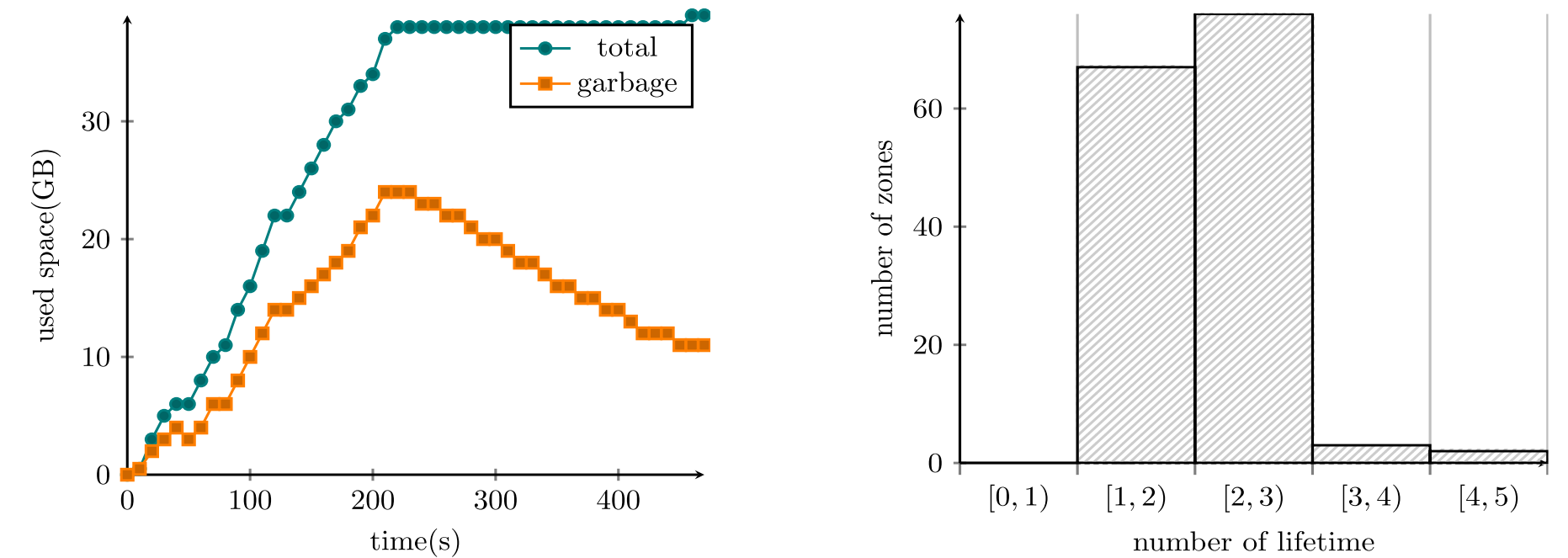


그림 5. 시간별 space usage

그림 6. zone 내의 lifetime 수에 따른 분포

- 200s 시점까지 총 사용량 선형적으로 증가, 그 후론 reclaim된 공간이 바로 재사용
- Reclaim 가능한 공간이 10GB 가량 있음에도 워크로드 추가 수행 불가
- 서로 다른 lifetime을 가진 데이터가 한 zone에 섞이는 것이 원인

연구 내용

SLA: Sst file Isolated Allocation

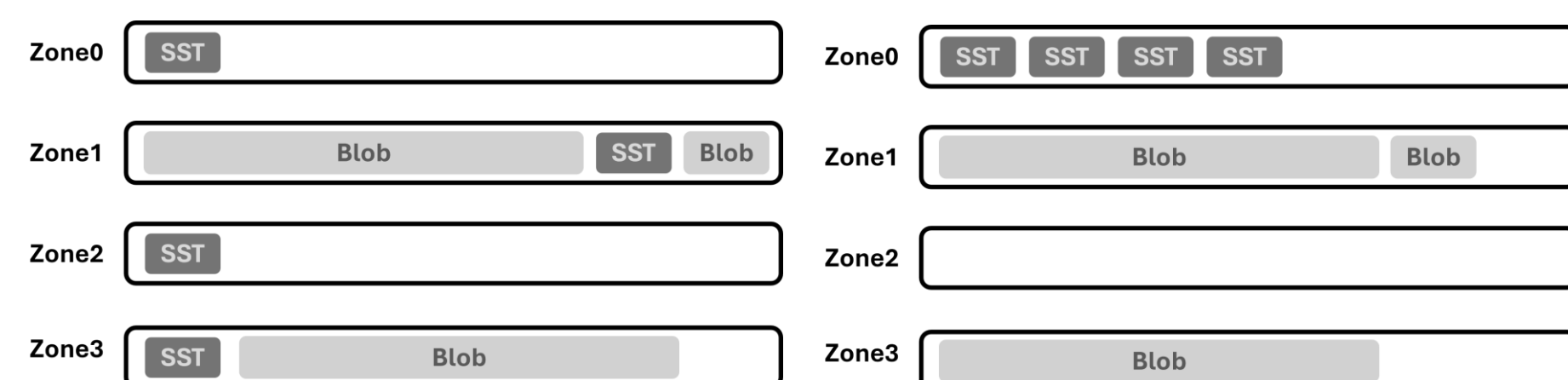


그림 7. 좌: 기존 ZenFS, 우: SIA

- Sst file을 별도 zone으로 분리
- Lifetime hint를 무시하고 가능한 한 같은 zone에 할당
- empty zone 사용량 감소

SLSIA: Strict Lifetime-aware SIA

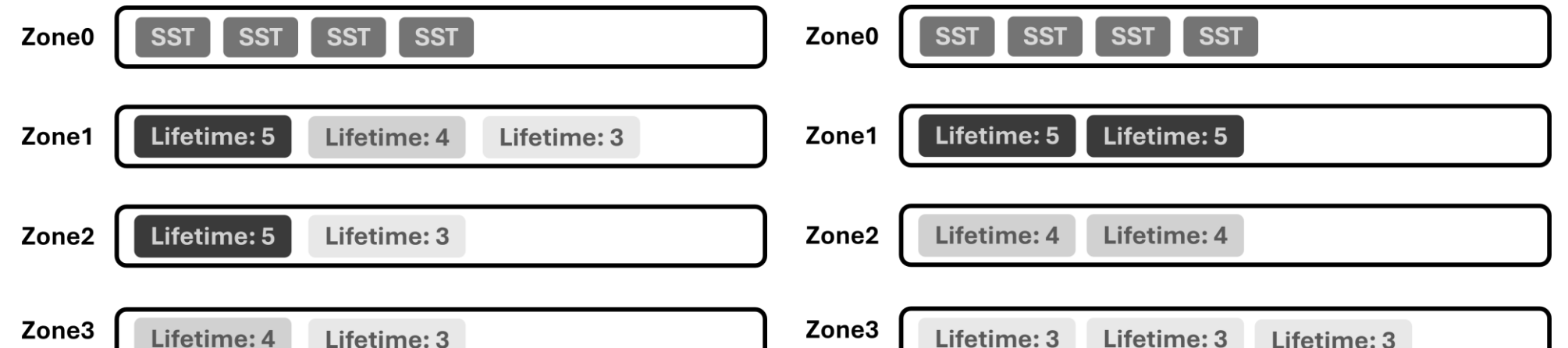


그림 8. 좌: SIA, 우: SLSIA

- SIA와 동시에, Blob file을 lifetime별로 같은 zone에 할당
- 더 효율적인 zone reclaim

연구 결과

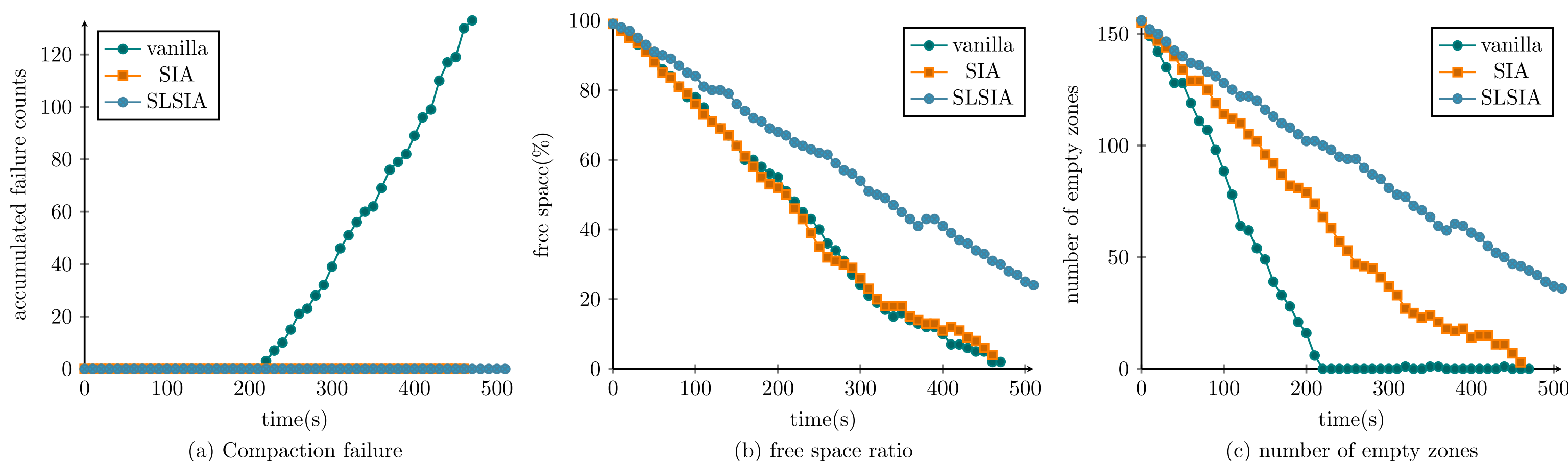


그림 9. 실험 결과

- SLA, SLSIA 모두 Zone 사용량 감소
- Compaction failure 문제 해결
- SLSIA의 효율적인 Zone 할당 전략
- 공간 사용량 9GB 가량 감소
- space utilization 72.9%→92.3%